UPSERT 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

있으면 업데이트, 없으면 삽입 — 두 작업을 하나로 합친 SQL.

---

1. 비유

출석부에 비유: - **INSERT**만: 이미 있는 학생을 또 추가 → 중복 에러 - **UPDATE**만: 없는 학생을 수정 → 아무 일도 안 일어남 - **UPSERT**: 있으면 수정, 없으면 추가 → 항상 올바른 결과

---

2. 개념

`INSERT ... ON CONFLICT ... DO UPDATE` 중복 키 충돌 시 에러 대신 UPDATE 실행.

---

3. 왜 쓰나

Dev 2에서 PR을 재스캔할 때: - 같은 `scan_id + agent` 조합의 집계가 이미 있을 수 있음 - 그냥 INSERT하면 UNIQUE 제약 위반 에러 - UPSERT로 안전하게 덮어쓰기

---

4. 코드 (Dev 2)

```
-- scan_summaries UPSERT
-- (scan_id, agent) 조합이 이미 있으면 수치 업데이트
INSERT INTO scan_summaries (scan_id, agent, critical, major, minor)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (scan_id, agent)          -- 이 조합이 겹치면
DO UPDATE SET                          -- INSERT 대신 UPDATE 실행
    critical = EXCLUDED.critical,      -- EXCLUDED = 방금 넣으려던 값
    major    = EXCLUDED.major,
    minor    = EXCLUDED.minor;
```

```
# db.py — Python에서 UPSERT 사용
await s.execute(text("""
    INSERT INTO scan_summaries (scan_id, agent, critical, major, minor)
    VALUES (:scan_id, :agent, :critical, :major, :minor)
    ON CONFLICT (scan_id, agent)
    DO UPDATE SET
        critical = EXCLUDED.critical,
        major    = EXCLUDED.major,
        minor    = EXCLUDED.minor
"""), {"scan_id": scan_id, "agent": agent, **summary})
```

복습 퀴즈

1. Q: UPSERT가 필요한 상황은? A: (같은 데이터를 여러 번 삽입할 수 있을 때. 있으면 업데이트, 없으면 삽입)
2. Q: `EXCLUDED`는 무엇을 가리키나? A: (충돌이 발생했을 때 INSERT하려던 새 값)
3. Q: Dev 2에서 UPSERT를 쓰는 이유는? A: (PR 재스캔 시 scan_summaries 중복 삽입 방지)